Migration Path Options Analysis

Cloudinary → UTA S3 (Enterprise External-Storage)

Project: AwsMigration — UTA Cloudinary → S3 Migration
Author: Edgar (Architect)
Date: March 15, 2026
Status: Draft — for Workshop with Client and Cloudinary Professional Services
Distribution: UTA Stakeholders, Cloudinary Professional Services

---

Purpose

This document presents a structured comparison of three candidate migration paths for moving ~3 TB of binary assets (images, videos, audio, PDFs) from Cloudinary-managed storage to UTA-owned S3, while retaining Cloudinary as the CDN and metadata management layer (BYOB — Bring Your Own Bucket).

The analysis is intended as a decision-support artifact for the workshop between UTA, the delivery team, and Cloudinary Professional Services. Several options carry open unknowns that must be clarified by Cloudinary PS during that session.

Prerequisite (applies to all options): A new Cloudinary environment must be created before any migration work begins. This is non-negotiable for protecting the live production environment. The new environment will also be the target for BYOB configuration.

The Three Candidate Options

Option A — Cloudinary CLI Sync / Clone

Use the Cloudinary CLI's `sync` or `clone` command to replicate assets and metadata from the existing Cloudinary environment into the new environment backed by UTA's S3 (BYOB).

- **`cld sync`**: Documented for local-folder <-> cloud synchronization. Whether it supports cloud-to-cloud (environment-to-environment) is **not confirmed** by public documentation.
- **`cld clone`**: Appears to perform environment-level duplication. Whether it carries binary assets, metadata, structured fields, and tags into a BYOB-backed target is **not confirmed**.
- If it works as hoped, this collapses the migration into a single command execution managed by Cloudinary's platform.

Reference: Cloudinary CLI sync docs | Cloudinary CLI clone docs

---

Option B — Worker Fleet (Python SDK + SQS + DynamoDB)

A purpose-built migration engine: a Python controller enqueues all asset identifiers into SQS; a pool of stateless workers reads the queue, downloads each asset from Cloudinary, uploads it to UTA S3, and updates Cloudinary metadata in the new environment. This is the architecture described in the PRD.

- Full programmatic control over every asset.
- Rate limiting, retry, and dead-letter handling are first-class concerns in the design.
- Per-asset state is tracked in DynamoDB; observability via CloudWatch and Datadog.
- Pilot batch (100 assets) is a native concept — easy to validate before scaling.

Reference: PRD Section "Worker Flow" and "Components."

Option C — S3 Tenant Ownership Transfer (or Cross-Account Copy)

Copy raw object data at the S3 layer — either from Cloudinary's internal S3 bucket directly to UTA's S3 bucket — and then reconcile Cloudinary metadata separately.

- Sub-variant C1: Ask AWS to perform a storage-layer "ownership transfer" (bucket ownership change across tenants). This is **not a standard AWS feature** and is considered highly unlikely.
- Sub-variant C2: Cloudinary configures a cross-account S3 replication or `CopyObject` permission from their internal bucket to UTA's S3. This is more realistic but depends entirely on Cloudinary PS willingness to expose their internal bucket.
- If feasible, this would be the fastest path for raw data transfer (server-side, zero egress cost).

Comparison Table

Quality Attribute	**A — CLI Sync / Clone**	**B — Worker Fleet (Python SDK)**	**C — S3 Ownership / Cross-Account Copy**
Feasibility (confirmed)	🟡 Partially unknown	✅ Fully understood	🔴 High uncertainty
Metadata Migration	🟡 Unknown (depends on CLI behavior)	✅ Explicit — per-asset API call	🔴 Not inherent — separate step required
`public_id` Preservatio	🟡 Assumed yes for `clone`; unclear for `sync`	✅ Guaranteed by design	🔴 Must be reconciled separately

Quality Attribute	A — CLI Sync / Clone	B — Worker Fleet (Python SDK)	C — S3 Ownership / Cross-Account Copy
n			
Admin API Rate Limit Exposure	🟡 Unknown — CLI may call Admin API internally	✅ Controlled — token-bucket rate limiter	🟢 Minimal — no Cloudinary API calls for transfer
Cloudinary PS Dependency	🔴 High — CLI behavior in cross-env + BYOB context must be validated by PS	🟡 Medium — PS scoped to BYOB setup only	🔴 Very high — non-standard AWS + Cloudinary operation
Transfer Speed (~3 TB)	🟡 Unknown — fast if cloud-native; slow if routed through client	🟡 Tunable via worker concurrency; bounded by API cap	🟢 Potentially fastest — server-side, same or nearby region
Data Egress Cost	🟡 Unknown — may incur if CLI routes through client	🟡 Incurred — download from Cloudinary + upload to S3	🟢 Potentially zero — if server-side S3 copy
Operational Visibility	🔴 Low — CLI is a black box	✅ Full — DynamoDB state, DLQ, CloudWatch, Datadog	🔴 Low — opaque AWS/Cloudinary-side operation
Per-Asset Retry / DLQ	🔴 Not available	✅ Native — exponential backoff + SQS DLQ	🔴 Not available
Rollback Capability	🟡 Moderate — new env isolated; partial-run state unclear	✅ Strong — DynamoDB snapshot per asset; explicit rollback path	🔴 Poor — reverting a cross-account copy or ownership transfer is complex
Risk to Live Environment	🟢 Isolated — new Cloudinary env only	🟢 Isolated — workers touch new env only	🔴 Potentially high — depends on AWS/Cloudinary infra boundaries
Pilot Batch Support (100 assets)	🟡 Can scope to a folder; hard to instrument	✅ Native — first-class feature in queue/worker model	🔴 Hard to pilot — likely all-or-nothing at bucket level
URL Continuity (CNAME)	✅ CNAME swap confirmed viable in Q&A 2	✅ Same — new env + same `public_id` + CNAME swap	🟡 Viable only if `public_id` reconciliation succeeds
Implementation Effort	🟢 Low (if CLI behaves as hoped)	🔴 High — full controller + worker +	🟢 Low (if supported) — primarily

Quality Attribute	A — CLI Sync / Clone	B — Worker Fleet (Python SDK)	C — S3 Ownership / Cross-Account Copy
		infra build-out	administrative
AWS + Infra Work Required	🟢 Minimal (S3 bucket + BYOB setup)	🟡 Moderate — ECS/Batch, SQS, DynamoDB, CloudWatch	🟢 Minimal — S3 bucket + cross-account policy

Legend: ✅ Confirmed favorable | 🟡 Uncertain / conditional | 🔴 Risk or blocker

---

Quality Attribute Summary (Architecture Lens)

Quality Attribute	Recommended Option	Rationale
Reliability	**B**	Full idempotency, per-asset DLQ, retry with backoff — designed for 3 TB at scale
Observability	**B**	Only option with per-asset state tracking, metrics pipeline, and alerting
Rollback Safety	**B**	DynamoDB snapshots + isolated new environment = safest rollback story
Lowest Engineering Effort	**A** (if CLI works) / **C** (if AWS supports it)	One command or one administrative operation
Lowest Data Transfer Cost	**C** (if feasible)	Server-side copy = zero egress
Fastest Transfer	**C** (if feasible)	No client-side bandwidth; otherwise **A** if CLI is cloud-native
Lowest PS Dependency	**B**	PS scope is well-defined (BYOB setup); migration is entirely self-directed
Lowest Risk Overall	**B**	All quality attributes are understood; no external blockers

---

Recommendation

Option B (Worker Fleet) is the recommended baseline and the fallback of last resort. It is the only option where: - All quality attributes are fully understood before work starts. - The

5,000 req/hr Admin API rate limit is respected by design. - UTA retains full operational control and observability. - A pilot batch (100 assets) validates the approach before full-scale execution. - Rollback is explicit and reversible at the per-asset level.

Option A (CLI) is the highest-ROI clarification item for the Cloudinary PS workshop. If `cld clone` replicates metadata and binary assets into a BYOB-backed new environment in a single operation, the engineering effort drops significantly and Option A becomes the preferred path.

Option C is worth raising but treat as low-probability. Sub-variant C2 (cross-account S3 copy via Cloudinary-configured bucket policy) is the only technically plausible form. Sub-variant C1 (AWS tenant ownership transfer) is not a supported AWS primitive.

Open Questions for Cloudinary Professional Services Workshop

The following questions must be resolved before selecting a migration path. These are the primary agenda items for the PS engagement session.

#	Question	Relevant Option	Impact if Unresolved
1	Does `cld clone` copy binary assets, metadata (tags, context, structured fields), and `public_id` exactly as-is into a new environment?	A	Cannot assess Option A feasibility
2	Does `cld sync` support cloud-to-cloud (environment-to-environment) operation, or is it limited to local-folder <-> cloud?	A	If local-only, Option A is eliminated
3	Does either CLI command consume Admin API quota internally, and what is the request rate behavior at scale?	A	Risk of exhausting 5,000 req/hr during migration
4	Will CLI commands operate correctly against a target	A	CLI + BYOB combination may not be validated by

#	Question	Relevant Option	Impact if Unresolved
	environment that already has BYOB (S3) configured as the storage backend?		PS
5	Can Cloudinary configure a cross-account S3 permission (replication rule or `CopyObject` IAM grant) from Cloudinary's internal bucket to UTA's S3 bucket?	C	Determines whether Option C is viable at all
6	What is the PS engagement scope, estimated hours, and timeline for BYOB configuration + new environment setup?	All	Required for project planning and budgeting
7	Is there a supported way to do a bulk metadata export (all `public_ids`, tags, context fields, resource types) from the current environment to use as input to a migration script?	B	Accelerates Option B controller implementation
8	Can the Admin API rate limit be temporarily raised to 20,000+ req/hr for the migration window, and what is the process to request it?	B	Determines max throughput and migration duration

Assumptions Carried Forward

- A new Cloudinary environment is mandatory before any migration activity.
- `public_id` values will be preserved exactly in the new environment (confirmed in Q&A 2).
- CNAME cutover approach is viable for URL continuity (confirmed in Q&A 2).
- PS handles BYOB backend configuration; UTA/delivery team handles data transfer and application validation.
- Cloudinary backup storage cannot be used as a migration source (confirmed in Q&A 2).
- Migration runs are scheduled outside West Coast peak hours (9 AM – 5 PM PT).
- Encryption at rest (SSE-KMS) is mandatory on the UTA S3 bucket; KMS key policy must permit Cloudinary's IAM principal.

Related Documents

- Cloudinary Q&A 1
- Cloudinary Q&A 2